UNIVERSIDAD ANDRÉS BELLO · UNAB ONLINE

Asignatura APTC101 · Programación orientada a objetos

Sistema de gestión de biblioteca

Sede Antonio Varas

Semana 6 · Sumativa 2: Trabajo grupal – Entrega final

Integrantes

N°	Nombre y apellido
1	Rommy Bejar Rivera
2	Francisco Pérez Salinas
3	Gonzalo Morgado Jofre
4	

Fecha: noviembre de 2026

Índice

1. Introducción.....3
2. Análisis y diseño de la solución.....4
2.1. Requerimientos y supuestos.....4
2.2. Clases, atributos y responsabilidades.....4
2.3. Diagrama de clases UML.....5
2.4. Fundamentación de las relaciones.....7
2.5. Relación con la entrega formativa (semana 4).....8
2.6. Análisis crítico del diagrama de clases.....8
3. Aspectos y fundamentos de programación.....9
3.1. Tecnología y estructura del proyecto.....9
3.2. Clases del modelo: atributos, constructores, mutadores, accesadores y toString.....9
3.3. Validaciones de las reglas de negocio.....9
3.4. Métodos de negocio: préstamo y devolución.....10
3.5. Colecciones y persistencia en archivos.....10
3.6. El método main.....10
3.7. Interfaz gráfica y navegación.....10
3.8. Buenas prácticas de programación.....11
3.9. Pruebas de funcionamiento.....11
4. Conclusiones.....12
5. Bibliografía.....13

1. Introducción

La biblioteca de la sede Antonio Varas de la Universidad Andrés Bello necesita administrar sus usuarios, sus libros y los préstamos y devoluciones de forma consistente. Cuando esta gestión es manual o fragmentada aparecen problemas conocidos: usuarios duplicados, errores en el inventario, libros prestados dos veces sin que nadie lo note y multas mal calculadas.

Motivación. El equipo de la biblioteca solicitó actualizar su sistema con requerimientos precisos (usuarios, libros, préstamos, devoluciones y almacenamiento en archivos de texto). El problema es un contexto laboral real, con reglas de negocio claras, y permite aplicar los fundamentos de la programación orientada a objetos (POO): clases, herencia, polimorfismo, encapsulamiento y distintos tipos de relaciones entre clases.

Objetivo del proyecto. Diseñar e implementar en Java una aplicación de escritorio con interfaz Swing (JFrame y JDialog) y almacenamiento en archivos de texto plano, capaz de registrar estudiantes y docentes, administrar los libros y controlar préstamos y devoluciones según las reglas del enunciado. Los objetivos específicos son: (1) modelar las entidades con un diagrama de clases UML y sus relaciones; (2) implementar las reglas de validación dentro de las propias clases, mediante constructores y mutadores; (3) construir una interfaz que permita volver siempre al inicio; (4) guardar y recuperar los datos en archivos de texto; y (5) comprobar los casos de éxito y de rechazo con un método main de demostración y con pruebas automáticas.

Objetivo de este informe. Presentar y fundamentar el diseño de la solución (clases, atributos, métodos y relaciones de asociación, agregación, composición, herencia y polimorfismo), explicar las decisiones de programación y mostrar los resultados de las pruebas.

Estructura. El capítulo 2 presenta el análisis y diseño, con el diagrama de clases, la fundamentación de las relaciones y un análisis crítico. El capítulo 3 explica los aspectos y fundamentos de programación: las clases, las validaciones, los métodos de préstamo y devolución, las colecciones y los archivos, el método main, la interfaz y las pruebas. El capítulo 4 contiene las conclusiones y el capítulo 5, la bibliografía.

2. Análisis y diseño de la solución

2.1. Requerimientos y supuestos

Del enunciado se identificaron los siguientes requerimientos funcionales:

- **Usuarios:** alta, edición y eliminación. Cada usuario tiene nombre completo, RUN, género, carrera y préstamo. El RUN no puede repetirse y se valida su formato y dígito verificador; el género debe ser M o F; el préstamo vale 0 si no tiene libro, o el ISBN si lo tiene. Los usuarios son estudiantes o docentes; a los docentes se les registra además la profesión y sus grados (magíster y/o doctor).
- **Libros:** ISBN único, título, autor, cantidad en biblioteca, cantidad disponible e imagen. La cantidad en biblioteca debe ser mayor a cero y la disponible debe ser mayor a cero y menor o igual a la anterior. Se pueden crear y eliminar (por ISBN, validando que exista).
- **Préstamo:** el libro y el usuario deben existir, el libro debe tener un ejemplar disponible y el usuario debe estar habilitado. La fecha de inicio es la actual, los días no pueden superar 20 (docente) o 10 (estudiante) y la fecha de devolución se calcula automáticamente. Se descuenta un ejemplar, el usuario queda no habilitado y se genera la tarjeta de préstamo para imprimir.
- **Devolución:** el libro y el usuario deben existir y el ISBN debe coincidir con el préstamo del usuario. Se suma un ejemplar, se habilita al usuario y se cobra una multa de $1.000 por cada día de atraso.
- **Operación:** menú principal con acceso permanente al inicio, interfaz con JFrame y JDialog, y todos los cambios guardados en archivos de texto plano.

Supuestos del grupo. (1) Un usuario solo puede tener un préstamo activo a la vez, porque el enunciado representa el préstamo con un único atributo (0 o ISBN). (2) La carrera es obligatoria para todos los usuarios, porque el enunciado la incluye entre los atributos comunes; en un docente se interpreta como la carrera o área en la que enseña. (3) Los grados del docente deben ser Magíster, Doctor o ambos. (4) El inventario de un libro ya prestado puede llegar a cero disponibles; la exigencia de «mayor a cero» se aplica al crear el libro.

2.2. Clases, atributos y responsabilidades

Se identificaron once clases, más los formularios JDialog. Todos los atributos son privados y se acceden por métodos públicos (encapsulamiento). Cada clase del modelo valida sus propios datos.

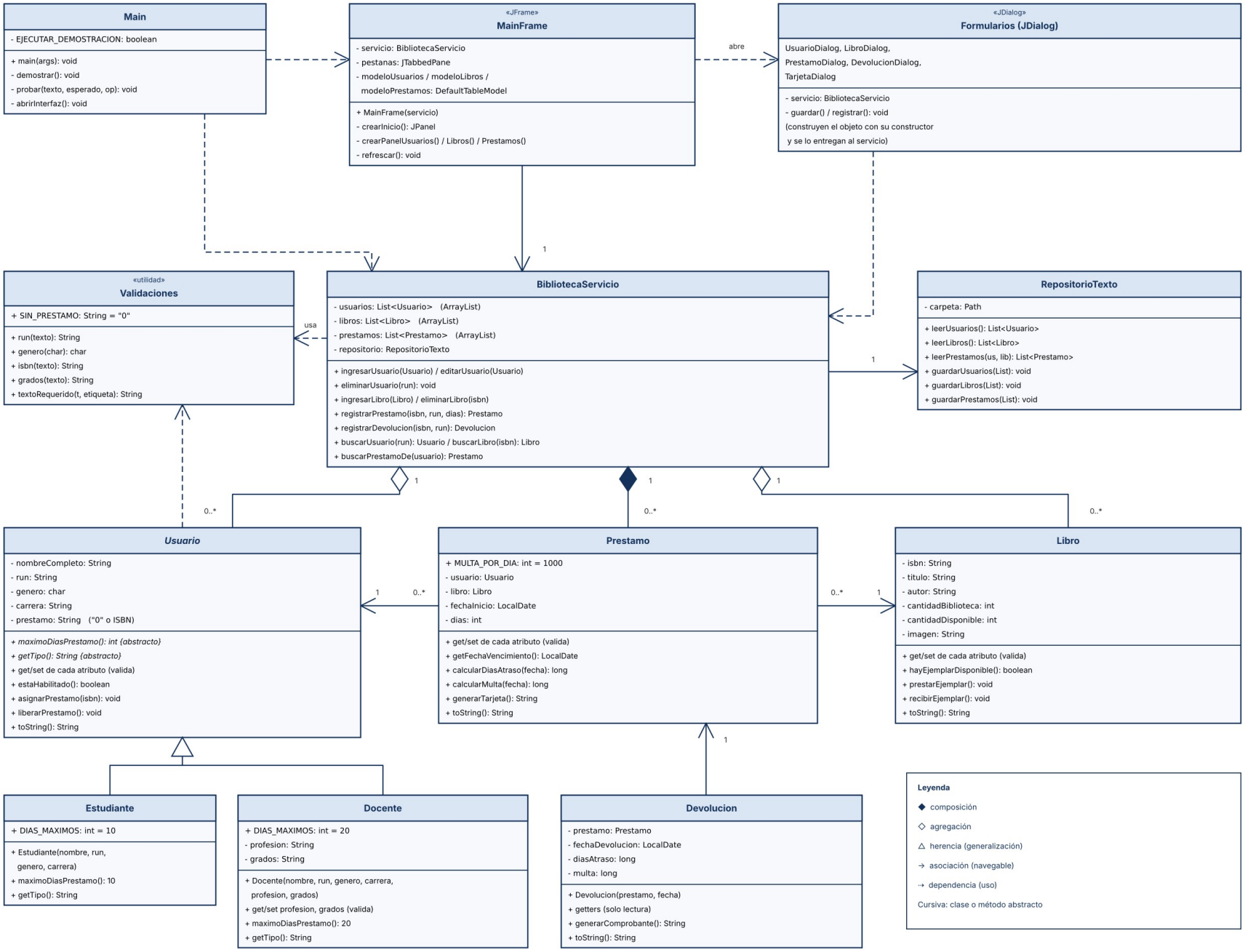
Clase	Responsabilidad	Atributos y métodos principales
Usuario (abstracta)	Datos comunes de docentes y estudiantes; declara el plazo máximo de forma polimórfica y controla su atributo préstamo.	- nombreCompleto, run, genero, carrera, prestamo + maximoDiasPrestamo(): int {abstracto} + getTipo(): String {abstracto} + estaHabilitado(), asignarPrestamo(isbn), liberarPrestamo()
Estudiante	Especialización de Usuario con plazo de 10 días.	+ Estudiante(nombre, run, genero, carrera) + maximoDiasPrestamo() → 10
Docente	Especialización de Usuario con plazo de 20 días, profesión y grados.	- profesion, grados + Docente(nombre, run, genero, carrera, profesion, grados) + maximoDiasPrestamo() → 20
Libro	Identidad por ISBN e inventario de ejemplares; decide si puede prestar o recibir un ejemplar.	- isbn, titulo, autor, cantidadBiblioteca, cantidadDisponible, imagen + hayEjemplarDisponible(), prestarEjemplar(), recibirEjemplar()
Prestamo	Préstamo activo; guarda referencias al usuario y al libro, y calcula vencimiento, atraso, multa y tarjeta.	- usuario, libro, fechaInicio, dias + getFechaVencimiento(), calcularDiasAtraso(f), calcularMulta(f) + generarTarjeta(): String

Clase	Responsabilidad	Atributos y métodos principales
Devolucion	Resultado de una devolución: préstamo cerrado, fecha, días de atraso y multa (solo lectura).	- prestamo, fechaDevolucion, diasAtraso, multa + generarComprobante(): String
BibliotecaServicio	Casos de uso: altas, bajas, préstamos y devoluciones; coordina modelo y archivos.	- usuarios, libros, prestamos (ArrayList), repositorio + ingresarUsuario(), editarUsuario(), eliminarUsuario() + ingresarLibro(), eliminarLibro() + registrarPrestamo(), registrarDevolucion()
RepositorioTexto	Lectura y escritura de usuarios.txt, libros.txt y prestamos.txt.	- carpeta: Path + leerX() y guardarX() para usuarios, libros y préstamos
Validaciones	Funciones estáticas de validación y normalización (RUN, género, ISBN, grados, textos).	+ run(), genero(), isbn(), grados(), textoRequerido(), enRango()
MainFrame y diálogos	Ventana principal (JFrame) y formularios JDialog (usuario, libro, préstamo, devolución, tarjeta).	- servicio + crear paneles, refrescar(); cada diálogo construye el objeto y llama al servicio
Main	Punto de entrada: demostración por consola y apertura de la interfaz.	+ main(args): void

2.3. Diagrama de clases UML

El diagrama completo se presenta en la página siguiente (y se entrega además como PDF independiente). Muestra las clases con sus atributos y métodos, las multiplicidades y los cinco tipos de relación que pide el enunciado.

Figura 1. Diagrama de clases final.



2.4. Fundamentación de las relaciones

La tabla resume cada relación del diagrama y la razón de haberla elegido. La notación sigue la especificación UML 2.5.1 (Object Management Group [OMG], 2017).

Tipo	Relación	Mult.	Justificación
Herencia	Estudiante y Docente ▷ Usuario	—	Relación «es-un»: un estudiante y un docente son usuarios. Heredan nombre, RUN, género, carrera y préstamo, y agregan lo propio (el docente, profesión y grados). Usuario es abstracta porque no existe un usuario «genérico» que pueda registrarse.
Polimorfismo	Usuario.maximoDiasPrestamo()	—	Prestamo.setDias() consulta usuario.maximoDiasPrestamo() sin preguntar el tipo: Estudiante devuelve 10 y Docente devuelve 20. Agregar un nuevo tipo de usuario no obliga a modificar el préstamo.
Asociación	Prestamo → Usuario	0..* a 1	Un préstamo conoce al usuario que lo solicitó mediante una referencia a su objeto. Cada usuario tiene a lo más un préstamo activo, pero el modelo no se limita a eso en el diagrama porque la regla se controla con el atributo préstamo del usuario.
Asociación	Prestamo → Libro	0..* a 1	Un préstamo recae sobre un libro, referenciado por su objeto. Un libro puede tener varios préstamos activos si tiene varios ejemplares.
Asociación	Devolucion → Prestamo	1 a 1	Una devolución conserva el préstamo que cierra para informar fechas, atraso y multa. El préstamo ya fue retirado de la lista activa, pero el objeto sigue existiendo.
Asociación	BibliotecaServicio → RepositorioTexto	1 a 1	El repositorio se crea en Main y se entrega al servicio por el constructor. El servicio lo usa pero no lo posee, por eso es asociación y no composición. MainFrame y los diálogos también se asocian al servicio que se les entrega.
Agregación	BibliotecaServicio ◇— Usuario	1 a 0..*	El servicio reúne usuarios, que son entidades con identidad propia (RUN) y se cargan desde un archivo; no nacen ni mueren con el servicio.
Agregación	BibliotecaServicio ◇— Libro	1 a 0..*	Igual que con los usuarios: el servicio reúne libros identificados por su ISBN, que existen independientemente de él.
Composición	BibliotecaServicio ◆— Prestamo	1 a 0..*	Un préstamo activo solo existe dentro del servicio: lo crea registrarPrestamo() y lo elimina registrarDevolucion(); nadie más lo crea ni lo comparte. Su ciclo de vida depende del todo.
Dependencia	Main ⇢ MainFrame, BibliotecaServicio; BibliotecaServicio y Usuario ⇢ Validaciones; MainFrame ⇢ diálogos; servicio ⇢ Devolucion	—	Relaciones de uso o creación sin guardar una referencia permanente: Main crea la ventana y el servicio; las clases usan los métodos estáticos de Validaciones; el servicio crea la Devolucion al registrar la devolución. Para no recargar el diagrama no se dibujan las flechas de uso de Libro, Docente y Prestamo hacia Validaciones.

Encapsulamiento. Los atributos de todas las clases son privados. Cada cambio de estado pasa por métodos de la propia clase: el inventario de un libro solo cambia con prestarEjemplar() y recibirEjemplar(), y el atributo préstamo de un usuario con asignarPrestamo() y liberarPrestamo(). El servicio coordina esos métodos y, si el guardado en archivos falla, vuelve a leer el disco para no dejar la memoria desfasada.

2.5. Relación con la entrega formativa (semana 4)

En la entrega formativa se modeló una biblioteca genérica con material bibliográfico abstracto (Libro y Revista), ejemplares, catálogo y préstamos, para practicar las cinco relaciones. Para esta entrega final se rediseñó el modelo a partir del enunciado oficial de la sede Antonio Varas, que exige otros conceptos: usuarios de dos tipos, libros con inventario, plazos, multas y almacenamiento en archivos.

Se conservaron los criterios de diseño de la formativa: una clase abstracta con herencia y polimorfismo (de MaterialBibliografico.calcularPlazo() a Usuario.maximoDiasPrestamo()), composición solo cuando hay dependencia de ciclo de vida, agregación para colecciones de entidades con identidad propia y una única fuente de verdad para el estado.

[Completar: observaciones del docente en la entrega formativa, si las hubo, y cómo se incorporaron.]

2.6. Análisis crítico del diagrama de clases

- **Usuario abstracta con dos subclases.** Estudiante y Docente comparten casi todos sus datos y solo difieren en el plazo y en la profesión y los grados. La herencia evita duplicar código y el polimorfismo elimina los condicionales por tipo (Fowler, 2004).
- **Reglas dentro de las clases del modelo.** Cada clase valida sus datos en el constructor y en los mutadores (RUN válido, género M/F, rangos de cantidades, plazo según el tipo de usuario). Así un objeto inválido no puede existir, sin importar quién lo cree: la interfaz, el main o la lectura de un archivo. El servicio queda para las reglas que involucran a varios objetos (RUN o ISBN repetidos, usuario habilitado, ejemplar disponible).
- **Prestamo guarda referencias a objetos.** En una versión anterior el préstamo guardaba solo el RUN y el ISBN. Se cambió a referencias a Usuario y Libro porque el modelo queda más fiel al dominio y la tarjeta se arma sin buscar en el servicio. Al leer los archivos se reconstruyen las referencias a partir del RUN y el ISBN guardados. La contrapartida es que ese enlace debe resolverse al cargar los datos y, si un archivo referencia a un usuario inexistente, la carga se rechaza.
- **Devolucion como clase propia.** El resultado de una devolución (fechas, atraso y multa) es un concepto del dominio. Modelarlo como una clase de solo lectura evita devolver un número suelto y permite generar el comprobante con la misma información.
- **Colecciones, composición y agregación.** Los préstamos se componen en el servicio porque no existen fuera de él. Los usuarios y libros se agregan porque tienen identidad propia. El repositorio es una asociación porque se inyecta desde afuera. Varios autores señalan que la agregación aporta poca semántica adicional a una asociación (Fowler, 2004); aquí se usa para expresar de forma explícita la idea de colección.

Limitaciones. (a) La carrera es obligatoria incluso para docentes, para respetar el enunciado. (b) Los tres archivos se guardan de forma independiente: cada uno se reemplaza de forma segura, pero no hay una transacción conjunta. (c) El ISBN se valida por estructura (10 o 13 caracteres), no por su dígito de control. (d) La imagen del libro se guarda como ruta y no se muestra en pantalla. (e) Un usuario solo puede tener un préstamo a la vez.

3. Aspectos y fundamentos de programación

3.1. Tecnología y estructura del proyecto

El proyecto está desarrollado en Java SE (compatible con Java 11 o superior), con la API Swing para la interfaz, java.time para las fechas y java.nio.file para los archivos. El código fuente está en BibliotecaUNAB/src/cl/biblioteca y se organiza en paquetes: modelo (Usuario, Estudiante, Docente, Libro, Prestamo, Devolucion), servicio (BibliotecaServicio), persistencia (RepositorioTexto), util (Validaciones), ui (MainFrame y los diálogos) y prueba (PruebaLogica). La clase Main es el punto de entrada. El proyecto se abre en Apache NetBeans con «Open Project» y se ejecuta con la clase principal cl.biblioteca.Main.

3.2. Clases del modelo: atributos, constructores, mutadores, accesadores y toString

Cada clase del modelo declara sus atributos como privados y ofrece un constructor con todos los datos, un accesador (get) y un mutador (set) por atributo, y un método toString que describe el objeto. Los constructores no asignan directamente: llaman a los mutadores, de modo que la validación se escribe una sola vez. Por ejemplo, en Usuario:

```
protected Usuario(String nombreCompleto, String run, char genero, String carrera)
{
    setNombreCompleto(nombreCompleto);   // texto obligatorio
    setRun(run);                         // formato y dígito verificador
    setGenero(genero);                   // solo M o F
    setCarrera(carrera);
    this.prestamo = Validaciones.SIN_PRESTAMO;   // "0": sin libro en su poder
}
```

Estudiante y Docente heredan de Usuario: el constructor de la subclase llama a super(...) y el Docente agrega profesión y grados con sus propios mutadores. Cada subclase sobrescribe maximoDiasPrestamo() y getTipo(), que en Usuario son abstractos (polimorfismo). Libro controla su inventario con los métodos prestarEjemplar() y recibirEjemplar(), y Prestamo guarda referencias a su Usuario y su Libro.

3.3. Validaciones de las reglas de negocio

Las reglas se programaron como funciones reutilizables y se aplican en el lugar que corresponde a cada una:

- **En Validaciones (funciones estáticas):** run() quita puntos y guion, valida el formato y el dígito verificador con módulo 11 (incluidos K y 0) y devuelve el RUN normalizado; genero() acepta solo M o F; isbn() exige 10 caracteres (el último puede ser X) o 13 dígitos; grados() acepta Magíster, Doctor o ambos; textoRequerido() rechaza textos vacíos.
- **En las clases del modelo (mutadores y constructores):** Usuario valida nombre, RUN, género, carrera y el atributo préstamo («0» o un ISBN válido). Libro exige cantidad en biblioteca mayor a cero y una cantidad disponible entre 0 y la anterior. Prestamo exige usuario, libro y fecha, y que los días estén entre 1 y el máximo del tipo de usuario.
- **En BibliotecaServicio (reglas entre varios objetos):** el RUN y el ISBN no se repiten, el libro nuevo debe tener al menos un ejemplar disponible, un usuario con libro en su poder no se puede eliminar, un libro con ejemplares prestados no se puede eliminar, y en el préstamo y la devolución se verifica el orden de los requerimientos 3.1 y 3.2 del enunciado.

Todas las reglas informan el error con una excepción (IllegalArgumentException o IllegalStateException) y un mensaje que la interfaz muestra al usuario.

3.4. Métodos de negocio: préstamo y devolución

Los métodos de negocio (la rúbrica los llama «customers» y los asocia al ingreso de un arriendo; en este sistema el arriendo es el préstamo de un libro) están en BibliotecaServicio y delegan cada paso a la clase que corresponde.

registrarPrestamo(isbn, run, dias): (1) busca el libro y verifica que exista; (2) verifica que tenga un ejemplar disponible; (3) busca al usuario y verifica que exista; (4) verifica que esté habilitado; (5) construye el Prestamo con la fecha actual y los días, que valida el plazo según el tipo de usuario y calcula el vencimiento; (6) llama a usuario.asignarPrestamo(isbn) y libro.prestarEjemplar(); (7) agrega el préstamo a la lista y guarda los archivos; (8) devuelve el préstamo, con el que se genera la tarjeta para imprimir.

registrarDevolucion(isbn, run): (1) verifica que el libro exista; (2) verifica que el usuario exista; (3) verifica que el ISBN coincida con el préstamo del usuario; (4) crea la Devolucion, que calcula los días de atraso y la multa de $1.000 por día; (5) llama a usuario.liberarPrestamo() y libro.recibirEjemplar(); (6) quita el préstamo de la lista y guarda los archivos; (7) devuelve la Devolucion, de la que se obtiene el comprobante.

Los demás métodos de negocio son ingresarUsuario(), editarUsuario(), eliminarUsuario(), ingresarLibro() y eliminarLibro(), que reciben los objetos ya construidos y validados.

3.5. Colecciones y persistencia en archivos

El servicio mantiene tres listas ArrayList (usuarios, libros y préstamos activos) y las recorre para buscar por RUN o ISBN. El repositorio lee cada archivo con listas ArrayList y construye los objetos con sus constructores, de modo que un archivo alterado a mano con datos inválidos se rechaza al cargar. Los préstamos se enlazan con los usuarios y libros ya cargados a partir del RUN y el ISBN.

Los datos se guardan en usuarios.txt, libros.txt y prestamos.txt, con un registro por línea, campos separados por tabulaciones y los caracteres especiales escapados. Cada archivo se escribe primero en un archivo temporal y luego reemplaza al original, para reducir el riesgo de corrupción si el programa se interrumpe. Los datos se leen al iniciar y se guardan en cada alta, edición, baja, préstamo o devolución.

3.6. El método main

El método main de la clase Main hace dos cosas. Primero ejecuta una demostración por consola con datos fijos, sin ingreso por teclado y sobre una carpeta temporal, para no tocar los datos reales. Después abre la interfaz gráfica con los datos de la carpeta datos. La demostración crea los objetos con sus constructores, siguiendo las relaciones del diagrama:

- Crea el repositorio y se lo entrega al servicio por el constructor (asociación).
- Crea un Estudiante y un Docente con sus constructores (herencia) y los ingresa al servicio.
- Crea libros y los ingresa al servicio (agregación).
- Llama a registrarPrestamo() y registrarDevolucion(), que crean el Prestamo y la Devolucion (composición y asociación).

Para cada operación imprime si se aceptó o se rechazó y el motivo, con casos válidos e inválidos de usuarios, libros, préstamos, devoluciones y archivos, y al final muestra el resumen «29 de 29 verificaciones se comportaron como se esperaba». La constante EJECUTAR_DEMOSTRACION permite desactivarla.

3.7. Interfaz gráfica y navegación

MainFrame hereda de JFrame y muestra una pantalla de inicio con acceso a Usuarios, Libros y Préstamos y devoluciones. El botón «Volver al inicio» está disponible en todo momento. Los formularios son JDialog modales (UsuarioDialog, LibroDialog, PrestamoDialog, DevolucionDialog y

TarjetaDialog). Cada diálogo construye el objeto con su constructor y se lo entrega al servicio; los errores de validación se informan con mensajes. La fecha del préstamo y la de devolución se generan automáticamente, y la tarjeta del préstamo y el comprobante de la devolución se muestran en un área de texto con la opción de imprimir.

3.8. Buenas prácticas de programación

- **Nombres y convenciones:** clases en PascalCase, métodos y atributos en camelCase y constantes en mayúsculas; los nombres describen su función (Martin, 2008).
- **Clases completas:** cada clase del modelo tiene atributos privados, constructor, accesadores, mutadores y toString.
- **Modularidad:** métodos pequeños con una sola tarea y un retorno específico (boolean para consultas, long para la multa, un objeto cuando se crea algo). Las validaciones están separadas en funciones reutilizables.
- **Responsabilidades separadas:** el modelo valida sus datos, el servicio coordina, el repositorio guarda y la interfaz solo captura y muestra.
- **Documentación y orden:** todas las clases y los métodos principales tienen comentarios Javadoc, el código está indentado, con una instrucción por línea y ordenado por paquetes.
- **Robustez:** los errores de las reglas se informan con excepciones y mensajes claros, y los archivos se escriben de forma segura.

3.9. Pruebas de funcionamiento

El proyecto incluye dos mecanismos de prueba. El método main ejecuta 29 verificaciones de demostración, y la clase PruebaLogica (paquete prueba) ejecuta 43 pruebas automáticas sobre las validaciones, las clases del modelo y el servicio, usando carpetas temporales. Todas resultaron correctas.

Prueba	Resultado obtenido
RUN duplicado o con dígito verificador incorrecto	Rechazado
Género distinto de M o F; docente con grados inválidos	Rechazado
ISBN de 10 o 13 caracteres válidos / ISBN de 11 caracteres	Aceptado / rechazado
Cantidad disponible mayor que la cantidad en biblioteca, o cantidad en biblioteca igual a cero	Rechazado
Estudiante pide 11 días / docente pide 25 días	Rechazado (máximos 10 y 20)
Docente pide 20 días	Aceptado (polimorfismo)
Préstamo válido	Registra el vencimiento, descuenta el ejemplar y deja al usuario no habilitado
Segundo préstamo del mismo usuario; libro sin ejemplares; libro inexistente	Rechazado
Devolución con un ISBN distinto al prestado	Rechazado
Devolución con 3 días de atraso	Multa de $3.000
Devolución a tiempo	Sin multa; repone el ejemplar y habilita al usuario
Eliminar usuario con préstamo o libro prestado	Rechazado
Reiniciar el servicio	Los usuarios, libros y préstamos se recuperan desde los archivos

Las pruebas automáticas cubren la lógica de negocio. La interfaz gráfica se comprobó abriendo y recorriendo las ventanas de usuarios, libros, préstamos y devoluciones.

4. Conclusiones

Resultados. Se diseñó e implementó una aplicación Java para la biblioteca de la sede Antonio Varas que cubre los requerimientos de usuarios, libros, préstamos y devoluciones, con interfaz Swing y almacenamiento en archivos de texto. El diseño aplica herencia (Usuario, Estudiante y Docente), polimorfismo (el plazo máximo de préstamo), asociación, agregación, composición y encapsulamiento, y deja las reglas de validación dentro de cada clase.

Nivel de logro de los objetivos. Se cumplieron el modelado con UML, la implementación de las validaciones en constructores y mutadores, la interfaz con JFrame y JDialog con acceso permanente al inicio, y la persistencia en archivos. El método main demuestra la creación de los objetos y los métodos de préstamo y devolución con casos válidos e inválidos, y las 43 pruebas automáticas confirman el comportamiento de la lógica de negocio.

Capacidad crítica. El diseño tiene límites que se identificaron: los tres archivos no se guardan en una transacción conjunta, el ISBN no verifica su dígito de control, un usuario solo puede tener un préstamo a la vez y la interfaz no tiene pruebas automáticas. Además, el diseño inicial guardaba claves en el préstamo y se cambió a referencias a objetos, lo que mejoró el modelo pero exige reconstruir esos enlaces al cargar los datos.

Marco de acción futura. Como trabajo futuro se propone migrar el almacenamiento a una base de datos con transacciones, validar el dígito de control del ISBN, guardar el historial de préstamos devueltos y las multas acumuladas, mostrar la imagen del libro, permitir varios préstamos por usuario y agregar pruebas automáticas de la interfaz.

5. Bibliografía

Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3.ª ed.). Addison-Wesley.

Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). Java Platform, Standard Edition 11: API specification (Swing, java.time, java.nio.file). https://docs.oracle.com/en/java/javase/11/docs/api/

Universidad Andrés Bello. (2026). APTC101, Semana 6: Sumativa 2, trabajo grupal entrega final [Instrucciones, enunciado y rúbrica de la actividad]. UNAB Online.